

Architecture Selection Statement:

"We chose Microservices Architecture for the Automated Rubric Assignment Evaluator System."

1. Architectural Style Trade-off Analysis

Architectural Pattern	Key Characteristics & Fit for Problem Statement #02	Verdict
Microservices Architecture	Deconstructs the system into loosely coupled, independently scalable services (UI Gateway, Workflow Queue, Sandbox Runner, Rubric Calculator, Plagiarism Analyzer). Critical for isolating untrusted student code execution and absorbing submission bursts.	Selected (Ideal)
Layered Architecture	Organizes components into Presentation, Business Logic, and Data layers. While offering good conceptual separation of concerns, untrusted student code execution within the monolithic runtime risks process crashes, blocking web requests.	Rejected
Client-Server Architecture	Centralizes grading logic on a single server with thick/thin clients. Causes severe bottlenecking when 100+ students submit code at assignment deadlines; lacks isolated fault containment for execution sandboxes.	Rejected

2. Technical Architectural Justification

1. Scenario-Related Reason 1 — Independent Horizontal Elasticity (NFR-001): Academic submissions exhibit extreme peak-load spikes around 11:59 PM deadlines. Microservices architecture allows the compute-heavy *Sandbox Execution Engine* and *Plagiarism & Static Analysis Service* to dynamically scale out across multiple worker nodes independently of the student upload portal, guaranteeing that batch processing queues never stall.

2. Scenario-Related Reason 2 — Complete Fault Domain Containment: Executing arbitrary student submissions introduces risks of infinite loops, memory leaks, fork-bombs, and unhandled compiler exceptions. Decoupling the execution runtime from the core *Rubric & Grading Service* ensures that a container crash or timeout will never terminate the overall system, corrupt the gradebook, or interrupt active peer evaluations.

3. Security Advantage (NFR-002) — Defense-in-Depth Sandbox Isolation: By isolating untrusted student code execution within dedicated, network-restricted container workers behind the *ISandboxRunner* interface, student code is physically partitioned from the persistent database (Gradebook & LMS Integration Service) and host filesystem. Malicious scripts cannot access database credentials or peer code directories.

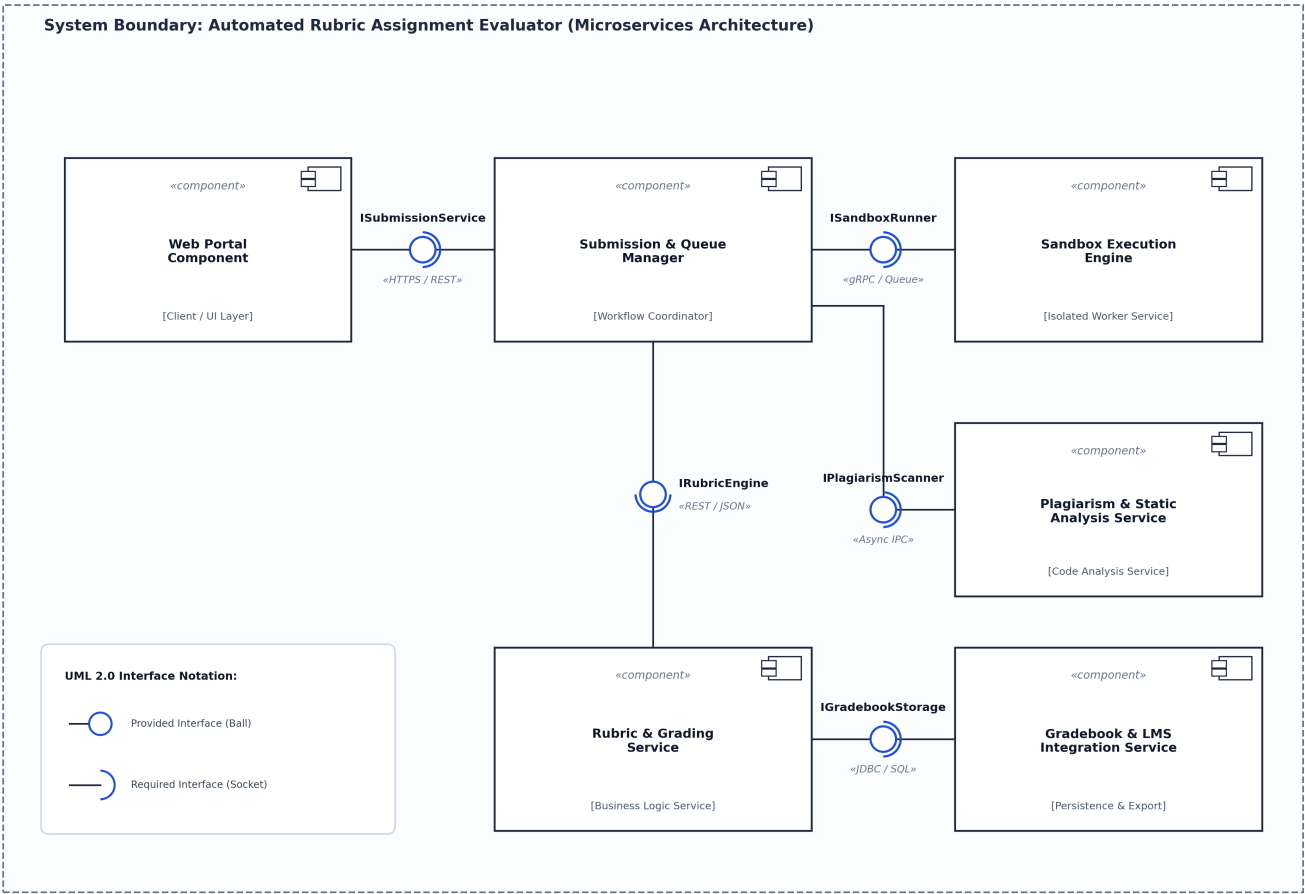
4. Performance Benefit — Asynchronous Event-Driven Decoupling: Heavy unit test suite execution and cohort plagiarism diff computations execute asynchronously via message queues. The student-facing *Web Portal* receives an immediate receipt ticket upon upload, enabling low-latency UI responsiveness without blocking HTTP connections during long test runs.

3. Component & Interface Specifications

Component Name	Stereotype	Provided Interface (Ball)	Required Interface (Socket)
Web Portal Component	«component» Client	—	ISubmissionService
Submission & Queue Manager	«component» Controller	ISubmissionService	ISandboxRunner, IRubricEngine, IPlagiarismScanner
Sandbox Execution Engine	«component» Worker	ISandboxRunner	—
Plagiarism & Static Analysis	«component» Worker	IPlagiarismScanner	—
Rubric & Grading Service	«component» Business	IRubricEngine	IGradebookStorage
Gradebook & LMS Integration	«component» Data	IGradebookStorage	—

4. UML 2.0 Component Diagram: Automated Rubric Assignment Evaluator

The diagram below illustrates the 6 architectural components, their internal boundaries, and the 5 assembly connectors using official UML 2.0 **Ball-and-Socket (Lollipop & Socket)** notation. Protocols and data-exchange mechanisms are explicitly labeled.



5. Interface Interaction & Data Flow Walkthrough

- Submission Ingestion:** The student uploads a project ZIP archive via the *Web Portal Component*. The portal invokes the *ISubmissionService* interface provided by the *Submission & Queue Manager* via «HTTPS / REST».
- Sandboxed Test Dispatch:** The *Submission & Queue Manager* parses the archive, checks deadlines, and queues an execution task across the *ISandboxRunner* interface (provided by *Sandbox Execution Engine* via «gRPC / Queue»). Code compiles and runs in an isolated container under a 5-second CPU limit.
- Static & Plagiarism Analysis:** In parallel, cohort source files are analyzed via the *IPlagiarismScanner* interface (provided by *Plagiarism & Static Analysis Service* via «Async IPC») to generate syntax checks and similarity diff reports.
- Rubric Computation:** Execution outcomes are forwarded to the *IRubricEngine* interface (provided by *Rubric & Grading Service* via «REST / JSON»), which applies instructor criteria weights (summing to 100%) and tallies peer evaluation scores.
- Persistent Storage & Export:** Finalized marks and faculty override remarks are persisted via the *IGradebookStorage* interface (provided by *Gradebook & LMS Integration Service* via «JDBC / SQL») and exported as LMS-compliant CSV files.